

INF672 · MACHINE LEARNING · UNIDAD 2

Práctica 1

Diez ejercicios sobre los fundamentos de la Unidad 2: estadística, vectores, probabilidad, preprocesamiento y gradiente

Este cuaderno reúne diez ejercicios de práctica sobre los fundamentos matemáticos y de datos de la Unidad 2. Cada uno es autocontenido: lee el **contexto**, sigue los **pasos** en orden y responde las preguntas del bloque **Para entregar**. Ninguno requiere entrenar un modelo todavía — trabajan la estadística, el álgebra vectorial, la probabilidad, el preprocesamiento y el descenso de gradiente que sostienen todo lo que viene después.

Para entregar tu trabajo desarrolla un notebook de Jupyter que incluya tanto el código ejecutado como tus respuestas escritas en celdas de texto, súbelo a tu repositorio de GitHub y envía el enlace a la plataforma virtual en el apartado de Práctica 1.

EJERCICIO 01

¿Cuánto sale par? Estima una probabilidad tirando el dado

Un dado tiene tres caras pares (2, 4, 6) de seis, así que sabes de antemano que la probabilidad de sacar par es $3/6 = 0.5$. En este ejercicio vas a *olvidar* ese dato y a estimarlo como si no lo conocieras: tirando muchas veces y contando. Es el enfoque frecuentista en su forma más simple.

1. Genera 1 000 lanzamientos de un dado con `rng.integers(1, 7, size=1000)`.
2. Cuenta cuántos resultados son pares usando una máscara booleana: `(tiradas % 2 == 0).sum()`.
3. Divide ese conteo entre el número de lanzamientos (N) para obtener tu estimación de la probabilidad (la frecuencia relativa).
4. Repite todo el proceso con 100 lanzamientos y luego con 100 000, guardando las tres estimaciones.

PARA ENTREGAR

Las tres estimaciones (con $N = 100$, 1 000 y 100 000) y una comparación de cada una con el valor teórico 0.5. Responde: ¿cuál se acercó más y por qué? ¿Qué le pasa a la estimación cuando tienes pocos datos?

EJERCICIO 02

Radiografía de una característica

Vas a describir con números una sola variable del dataset Iris, sin cruzarla todavía con ninguna otra. Elige el largo del pétalo (`petal_length`).

1. Carga Iris y extrae la columna del largo del pétalo.
2. Calcula cinco medidas con NumPy: media (`np.mean`), mediana (`np.median`), desviación estándar (`np.std`), mínimo (`np.min`) y máximo (`np.max`).
3. Escribe **una frase por cada medida** explicando qué te dice sobre los pétalos. Por ejemplo, ¿qué significa que la desviación estándar sea grande o pequeña frente a la media?

PARA ENTREGAR

Los cinco valores y las cinco frases de interpretación. Responde además: ¿la media y la mediana están cerca o lejos entre sí? ¿Qué sugiere eso sobre la forma de la distribución?

EJERCICIO 03

¿Qué tan lejos están dos flores?

Cada flor de Iris es un vector de cuatro números (largo y ancho de sépalo y pétalo). “Parecerse” entre flores se puede medir con la distancia euclidiana: flores parecidas están cerca, flores distintas están lejos.

1. Carga Iris y elige tres flores: **dos de la misma especie y una de otra especie**. Anota los índices que usaste.
2. Trata cada flor como un vector de cuatro componentes y calcula la distancia euclidiana entre cada par, de dos formas que deben coincidir:
 - a mano: `np.sqrt(np.sum((a - b)**2))`
 - con la función de NumPy: `np.linalg.norm(a - b)`
3. Compara la distancia entre las dos flores de la misma especie con la distancia hacia la flor de la otra especie.

PARA ENTREGAR

Las distancias calculadas por ambos métodos (verificando que dan lo mismo) y tu comparación. Responde: ¿la distancia entre flores de la misma especie resultó menor? ¿Por qué tiene sentido que la cercanía numérica refleje la especie?

EJERCICIO 04

Limpiar una tabla, paso a paso

Antes de que cualquier modelo pueda usar unos datos, hay que dejarlos limpios: sin huecos y sin texto. Vas a practicar los tres gestos básicos por separado sobre esta pequeña tabla. Créala en un DataFrame de Pandas:

ingreso	ciudad	edad
3200	Potosí	28
4100	Sucre	35
(vacío)	Potosí	41
2800	La Paz	23
5000	(vacío)	52
(vacío)	Sucre	30
3600	Potosí	45
4800	La Paz	38

1. **Detecta los faltantes:** cuenta cuántos valores vacíos hay por columna con `df.isna().sum()` e indica en qué columna hay más.
2. **Imputa:** rellena los faltantes de la columna `ingreso` con su mediana usando `df['ingreso'].fillna(df['ingreso'].median())`. Explica por qué se prefiere la mediana a la media cuando puede haber ingresos muy altos o muy bajos.
3. **Codifica:** convierte la columna `ciudad` en variables numéricas con `pd.get_dummies(df, columns=['ciudad'])` y observa cómo una columna de texto se transforma en varias columnas de ceros y unos.

PARA ENTREGAR

La tabla final, ya sin faltantes y sin texto, más tus dos explicaciones (por qué la mediana, y qué representa cada columna nueva creada por la codificación).

EJERCICIO 05

¿Se mueven juntas dos variables?

Dos características pueden estar relacionadas: cuando una crece, la otra también. Vas a detectar esa relación primero con los ojos y luego con un número. Usa el largo (`petal_length`) y el ancho (`petal_width`) del pétalo de Iris.

1. Dibuja un histograma del largo del pétalo con `plt.hist` para ver la forma de esa variable.
2. Dibuja un diagrama de dispersión de las dos variables con `plt.scatter` (largo en un eje, ancho en el otro) y observa si los puntos suben juntos.
3. Cuantifica esa impresión visual calculando el coeficiente de correlación de Pearson con `np.corrcoef(largo, ancho)[0, 1]`.

PARA ENTREGAR

Los dos gráficos y el valor de correlación. Responde: ¿el número coincide con lo que muestra la nube de puntos? Interpreta su signo y su magnitud — ¿qué significaría un valor cercano a +1, a 0 o a -1?

EJERCICIO 06**Perfil estadístico por especie**

Hasta ahora describiste variables una por una. Aquí vas a comparar las tres especies de Iris entre sí, calculando el perfil estadístico de cada una para descubrir cuál característica las distingue mejor.

1. Carga Iris en un DataFrame de Pandas, asegurándote de tener la columna de especie junto a las cuatro características.
2. Agrupa por especie con `df.groupby('species')` y calcula, para las cuatro características, estas medidas: media, mediana, desviación estándar, rango (máximo menos mínimo) y los cuartiles (Q1, Q2, Q3). Puedes apoyarte en `.agg([...])` o en `.describe()` y completar lo que falte.
3. Ordena los resultados en una tabla legible por especie y característica.

PARA ENTREGAR

La tabla de estadísticos por especie y una respuesta argumentada: ¿qué característica separa mejor a las tres especies y por qué? Fíjate en cuál tiene medias muy distintas entre especies **y** poca dispersión dentro de cada una — esa combinación es la que produce buena separación.

EJERCICIO 07**Tres formas de comparar dos flores, hechas por ti**

Vas a programar desde cero las tres operaciones con las que se compara qué tan parecidas son dos flores, usando solo NumPy y sin ninguna librería de machine learning. Así entenderás qué hace cada una por dentro antes de usarlas ya empaquetadas.

1. Implementa tres funciones propias que reciban dos vectores **a** y **b**:
 - distancia euclidiana: `np.sqrt(np.sum((a - b)**2))`
 - producto punto: `np.sum(a * b)`
 - similitud coseno: el producto punto dividido entre el producto de las normas, `np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))`
2. Elige cuatro flores de Iris: **dos de la misma especie** y **dos de especies distintas**. Anota los índices que usaste.
3. Aplica tus tres funciones a los pares que formes y organiza los resultados para poder compararlos.

PARA ENTREGAR

El código de las tres funciones y una tabla con los valores obtenidos para cada par. Responde: ¿la similitud coseno y la distancia euclidiana ordenan las flores igual (las mismas resultan "más parecidas")? ¿En qué caso discrepan, y por qué crees que ocurre —recordando que el coseno mira el ángulo y la euclidiana mira la separación?

EJERCICIO 08**Leer los datos con la vista**

Un buen análisis empieza mirando. Vas a producir tres vistas clásicas del dataset Iris y, sobre todo, a *interpretarlas* por escrito: el objetivo no es solo generar gráficos bonitos, sino leer qué características separan las especies y cuáles se confunden.

1. Construye un **pairplot** coloreado por especie con `seaborn.pairplot(df, hue='species')` para ver todas las combinaciones de características a la vez.
2. Construye un **mapa de calor de correlaciones** con `seaborn.heatmap(df.corr(numeric_only=True), annot=True)` para ver qué características se mueven juntas.
3. Construye **boxplots por característica** separados por especie (uno por cada una de las cuatro características) para comparar sus distribuciones.

PARA ENTREGAR

Los tres gráficos y una lectura escrita. Identifica qué pares de características logran una separación lineal visible entre las especies (donde podrías trazar una raya que las divida) y cuáles se solapan. Menciona al menos un par que separe bien y uno que confunda.

EJERCICIO 09**Bajar la colina: descenso de gradiente a mano**

Muchos modelos aprenden buscando el punto más bajo de una función de error. Aquí vas a implementar esa búsqueda tú mismo sobre una función sencilla cuyo mínimo ya conoces, para ver el mecanismo con claridad. La función es $f(x) = (x - 3)^2 + 2$, una parábola con mínimo en $x = 3$.

1. Define $f(x)$ en Python y aproxima su derivada numéricamente por diferencias finitas: `(f(x + h) - f(x - h)) / (2*h)` con un h pequeño, por ejemplo $1e-5$.
2. Programa un bucle de descenso de gradiente: parte de un x inicial (por ejemplo $x = 10$) y, en cada iteración, actualiza $x = x - tasa * derivada(x)$. Repite por un número fijo de pasos y guarda cómo evoluciona x .
3. Ejecuta el bucle con distintas tasas de aprendizaje: una pequeña (0.01), una intermedia (0.1) y una grande (1.1).

PARA ENTREGAR

La trayectoria de x para cada tasa y una observación escrita. Responde: ¿todas llegan cerca de $x = 3$? ¿Qué pasa con la tasa muy pequeña (¿avanza lento?) y con la muy grande (¿se pasa, oscila o se dispara)? Explica por qué la elección de la tasa es delicada.

EJERCICIO 10**Contar y simular: dos caminos hacia una probabilidad**

Vas a calcular una probabilidad de dos maneras y comprobar que coinciden: primero de forma exacta enumerando todos los casos posibles (enfoque clásico), y luego estimándola con simulación (enfoque frecuentista). Elige **uno** de estos eventos: “al menos un 6 al tirar cuatro dados” o “la suma de tres dados es 10”.

1. **Por conteo (exacto):** enumera el espacio muestral completo con `itertools.product(range(1, 7), repeat=n)` (con $n = 4$ o $n = 3$ según tu evento), cuenta cuántos resultados cumplen la condición y divide entre el total. Ese es el valor teórico.
2. **Por simulación (estimado):** simula muchas tiradas con `rng.integers(1, 7, ...)`, verifica el evento en cada una y calcula la frecuencia relativa.
3. **Convergencia:** grafica cómo la estimación se acerca al valor teórico a medida que aumentan las tiradas, y tabula el error ($|\text{estimación} - \text{valor exacto}|$) para $N = 10, 100, 10\,000$ y $100\,000$.

PARA ENTREGAR

El valor exacto por conteo, el gráfico de convergencia con la línea del valor teórico marcada, y la tabla de error por tamaño de muestra. Responde: ¿a partir de qué N la estimación se vuelve confiable? ¿Qué relación notas entre el número de tiradas y el error?